Pregunta 0.1 ¿Qué diferencia hay entre una clase y un objeto?
Una clase es como un plano o molde que sirve para crear objetos. Un objeto es algo que se crea a partir de esa clase y puede tener sus propios valores.

Pregunta 0.2 ¿Qué es la firma de un metodo y qué partes la forman?
La firma de un método es su nombre y los tipos de datos que recibe como parámetros, en el orden en que aparecen. No incluye el tipo de dato que devuelve
Ej. int sumar(int a, int b)
Su firma es sumar(int, int)

Pregunta 0.3 ¿Qué diferencia hay entre una variable local y un atributo de instancia?
Una variable local se declara dentro de un método y solo existe mientras se ejecuta ese método. Se le tiene que dar un valor antes de utilizarla.
Un atributo se declara dentro de la clase, pero fuera de los metodos. Tiene un valor inicial por defecto si no se le asigna uno y existe mientras exista el objeto.

Pregunta 0.4 ¿Qué significa null en una referencia?
null significa que una variable no está apuntando a ningún objeto. Si intentamos utilizar un método con esa variable, el programa marca un error llamado NullPointerException.

Pregunta 0.5 ¿Cuándo conviene usar for-each y cuándo un recorrido con índice?
El for-each se utiliza cuando queremos recorrer todos

los elementos de un arreglo y no necesitamos saber sus posiciones

Un for con índice sirve cuando necesitamos saber en qué posición está cada elemento o trabajar con una posición específica.

R1 • ¿Qué imprime?

```
StringBuilder ruta = new StringBuilder ("5");
StringBuilder alias = ruta;
for (int i=1; i<=3; i++) }
   alias.append (i/2);
   System.out.println (ruta);
{
String texto = "5" +2 + 1;
System.out.println (texto);
double mitad = (double)(7/2);
System.out.println (mitad);
```

Salida correcta:

```
50
501
5011
5021
 3.0
```

Tabla de seguimiento

Paso	i	i/2	ruta	Lo que imprime
Inicio	–	–	5	Nada
Vuelta 1	1	0	50	50
Vuelta 2	2	1	501	501
Vuelta 3	3	1	5011	5011
Texto	–	–	5011	521
Mitad	–	7/2=3	5011	3.0

Explicación
La variable "alias" apunta al mismo objeto que "ruta", por es cuando modificamos una también cambia lo que vemos en la otra.
En el ciclo se divide ":" entre 2, pero como son enteros, no se guardan los decimales.
Después, "5" + 2 + 1 da "521" porque se están juntando los valores como texto.
Finalmente, 7/2 da 3 porque son enteros. Después se convierte a "double", por eso imprime "3.0".

R2. ¿COMPILA, TRUENA O FUNCIONA?

a)

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

No compila.
En "int unidades = medida;" estamos intentando guardar un número decimal en una variable entera. Java no permite hacerlo directamente porque se perdería la parte decimal.
Para que funcione, tendría que ser "int unidades = (int) medida;" e imprimiría "6"

b)

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Compila, pero truena
El arreglo tiene tres elementos, pero sus posiciones son:

indice	Valor
0	2
1	5
2	8

La variable "indice" vale 3 porque "canales.lenght" devuelve el número de elementos.
El error es 'System. out.println (canales[indice]);
Porque intenta acceder a la posición 3, que no existe.

c) Funciona
El primer resultado es "false" porque == compara si las dos referencias apuntan al mismo objeto. Como utilizamos "new String", se crea otro objeto.
El segundo resultado es "true" porque ".equals()" compara lo que está escrito dentro de los textos y ambos contienen "F2"

B3 Modelado
Código corregido

```
public class Multimetro{
    final int inventario;
    String responsable;
    static fnl totalPrestados =0;
    public Multimetro (int numero){
        inventario = numero;
        responsable = null;
    }
    void prestar (String persona){
        responsable = persona;
        totalPrestados ++;
```

```
    {
    String consultarResponsable() }
        return responsable;
    {
}
```

Justificaciones

1. final int inventario
   Utilicé "int" porque el inventario es un número entero y "final" porque ese número no debe cambiar. Cada multímetro tiene su propio número de inventario.
2. String responsable
   Utilicé String porque se guarda el nombre de una persona. No es "final" porque el responsable puede cambiar
3. static int totalPrestados
   Queremos contar los préstamos de todos los multímetros, no de uno solo. Es "int" porque contamo números enteros.
4. Constructor
   Sirve para darle un número de inventario al multímetro cuando se crea. También le asigna "null" al responsable porque al principio no está prestado.

R4 Código con repeticion

```
static int contarDigitos(int folio)}
    int contador = 0;
    if (folio == 0)}
        return 1;
    {
    while (folio > 0)}
        folio = folio / 10;
        contador ++;
```

```
{
  return contador;
}
```

Tabla

Vuelta	folio antes de dividir	folio/10	contador
Inicio	4072	—	0
1	4072	407	1
2	407	40	2
3	40	4	3
4	4	0	4

RS Código con selección

```
static String nivelCarga(int carga){
    if (carga <0|| carga >50) {
        returno "INVALIDA";
    {
  switch (carga/10){
      case 0;
      case 1;
          return "BAJA";
      case 2;
      case 3;
          returno "MEDIA";
      case 4;
      case 5;
          return "ALTA";
      default;
          return "INVALIDA";
      {
  {
```

{ sirve para comprobar si la carga es menor que cero o mayor que 50